

R2D-RL: A RoboCup 2D Soccer Environment for Multi-Agent Reinforcement Learning

Haobin Qin¹, Baofeng Zhang¹, Hidehisa Akiyama², and Keisuke Fujii^{*1}

¹Graduate School of Informatics, Nagoya University, Furo-cho, Chikusa, Nagoya, Aichi, Japan

²School of Information and Data Sciences, Nagasaki University, Nagasaki, Japan

Preprint version, June 2026

Preprint notice: This manuscript is a preprint and has not been peer-reviewed or accepted for publication.

Abstract

Robot soccer is a challenging testbed for multi-agent reinforcement learning because it combines partial observability, cooperative and adversarial interaction, sparse rewards, and long-horizon tactical behavior. RoboCup 2D Soccer Simulation (RCSS2D) provides a mature robot-soccer platform, but its competition-oriented server-client architecture is difficult to use directly with modern Python-based MARL workflows. We introduce R2D-RL, a reinforcement learning environment that connects RCSS2D and HELIOS-based player clients to a Python MARL interface through shared-memory communication and cycle-level synchronization. R2D-RL supports full-field and scenario-based training with configurable opponents, Base discrete and Hybrid parameterized action spaces, action masks, expected possession value (EPV)-based reward shaping, and parallel execution. We provide front-goal scenarios and an 11-vs-11 full-field benchmark, together with baseline results.

Keywords: Multi-agent reinforcement learning, RoboCup 2D Soccer Simulation, robot soccer, action masks, reinforcement learning environment.

Funding: This work was financially supported by JST SPRING, Grant Number JPMJSP2125; NEDO Intensive Support Program for Young Promising Researchers, Grant Number 24021654; JSPS KAKENHI Grant Number 23H03282.

1 Introduction

Soccer is a representative domain for learning-based sports analytics, where strategic evaluation and next-play decision making require reasoning about many interacting players over space and time [1]. It combines partial observability, simultaneous cooperative and adversarial interaction, sparse scoring events, and long-horizon tactical organization. These properties make football a natural domain for studying multi-agent reinforcement learning and tactical decision making.

^{*}Corresponding author: fujii@i.nagoya-u.ac.jp

To support football reinforcement learning and multi-agent decision research, different types of football simulation and learning platforms have been proposed. One line of work is oriented toward football decision making and MARL benchmarks, where football is organized as a learning problem for algorithm evaluation, reward design, scenario-based learning, or large-scale training; representative examples include Google Research Football and NFootball [2, 3]. Another line of work is oriented toward robot soccer and embodied control, where football interaction is coupled with physical movement, low-level control, simulator protocols, and competition-style multi-agent interaction, with RoboCup being the most representative example.

RoboCup 2D Soccer Simulation (RCSS2D) [4, 5] represents a long-standing platform in this control-oriented robot-soccer tradition. It provides a full-field match setting with game rules, referee control, physics, stamina, noisy sensing, and a distributed server-client architecture. Around this simulator, the RoboCup 2D community has accumulated mature base teams and tactical behavior modules. In particular, HELIOS-based teams provide rule-based soccer behaviors such as passing, dribbling, shooting, interception, positioning, and set plays.

Several works have made parts of the RCSS2D ecosystem easier to use for learning research. At the client-team level, Pyrus and the Cross Language Soccer Framework simplify the development of RoboCup 2D rule-based client teams by providing Python or language-agnostic interfaces for simulator communication [6, 7]. Keepaway and Half Field Offense (HFO) provide RCSS2D-derived subtask simulation environments that serve as testbeds for algorithms studying local possession, attacking behavior, and multi-agent cooperation [8–10].

As reinforcement learning and multi-agent learning have developed, football simulation platforms are increasingly expected not only to run matches, but also to be organized as environments suitable for learning algorithms, for example by providing standardized step-based interaction, reproducible task control, training signals, parallel sampling, and benchmark settings. Google Research Football has attracted broad attention in football reinforcement-learning research in recent years precisely because it matches these requirements. By contrast, although RCSS2D provides full-field match mechanics and the HELIOS behavior ecosystem, it lacks a synchronization interface for modern MARL training, making it difficult to use directly for full-field reinforcement-learning research.

This paper presents R2D-RL, a Python MARL environment for RoboCup 2D. Rather than serving as another agent-development interface or reducing RoboCup 2D to a localized subtask, R2D-RL builds on the original RCSS2D server-client match process and HELIOS behavior modules, and adds a Python-side MARL interface for cycle-synchronized learning. Through shared-memory communication and cycle-level synchronization, the environment wraps the existing server and modified HELIOS clients, converting the competition workflow into a controllable learning interface. It provides full-field and scenario-based benchmarks, configurable opponents, Base discrete and Hybrid parameterized action spaces, controllable agents, action masks, and parallel execution.

The main contributions are as follows:

1. We design a shared-memory communication and cycle-level synchronization architecture that connects the original RCSS2D server-client workflow with Python MARL training and organizes it as step-synchronous reinforcement-learning interactions.
2. We provide a Python MARL interface that exposes local observations, centralized state, action masks, rewards, termination signals, and reset operations for learning algorithms.
3. We construct an R2D-RL benchmark suite covering front-goal scenarios and 11-vs-11 full-field learning.

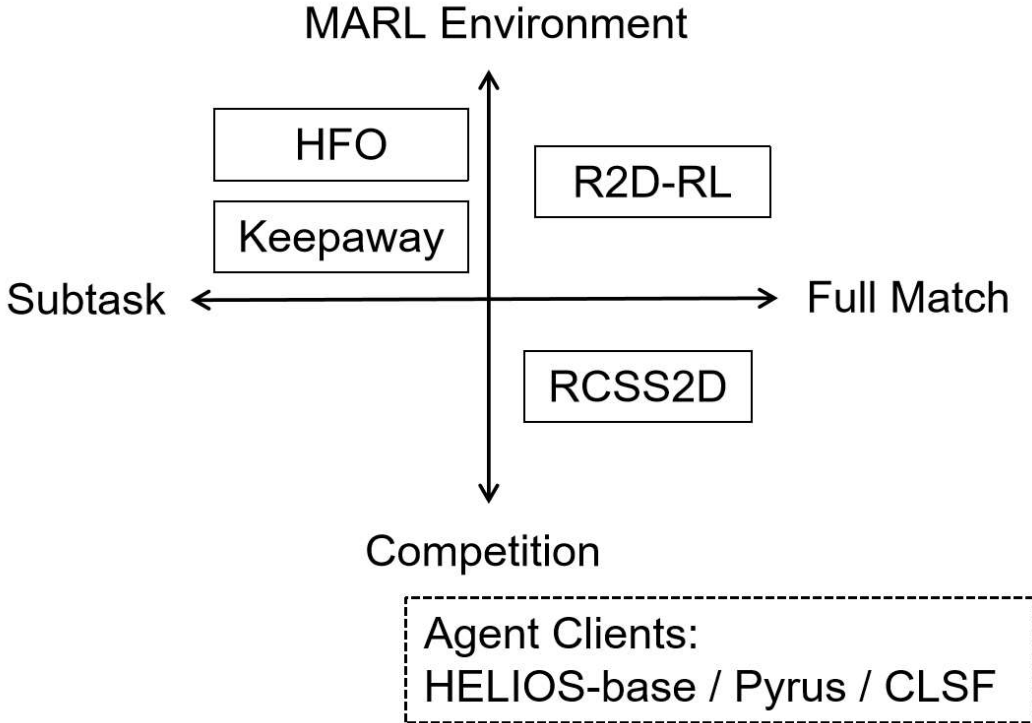


Figure 1: Positioning of R2D-RL in the RoboCup 2D ecosystem.

4. We define two policy-level action spaces: a parameterized Hybrid interface over selected RCSS2D body commands and a HELIOS-derived Base discrete action space with corresponding action masks.

2 Related Work

2.1 RoboCup 2D Soccer Simulation and Its Ecosystem

RoboCup is a long-running research initiative in robotics and artificial intelligence, originally proposed by Kitano et al. as a challenge problem for intelligent agents in a dynamic, real-time, multi-agent domain [11]. Within RoboCupSoccer, the Simulation League is one of the oldest branches and is focused on artificial intelligence and team strategy [12]. RoboCup 2D Soccer Simulation (RCSS2D) is a classic platform in this line of work. In the 2D Simulation League, two teams of eleven autonomous software agents play soccer in a two-dimensional virtual stadium represented by a central SoccerServer [4, 5, 13]. The server maintains the players, ball, physics, and match state; player clients issue commands from local perceptions; the coach provides global match information; and the trainer supports reset and scenario control.

Full-field RCSS2D games involve long horizons, sparse scoring events, and many simultaneously acting agents. Against this background, several lightweight RCSS2D-derived subtask environments have been introduced to support learning-algorithm development and testing. Keepaway simplifies football into a small-sided ball-possession task and has been used as a testbed for algorithms studying high-level decisions such as holding the ball, passing, and maintaining space under multi-agent interaction [8, 9]. Half Field Offense (HFO) restricts the problem to a half-field attacking

environment with clearer state, action, and reward interfaces for testing algorithms for possession, passing, shooting, attacking cooperation, and multi-agent reinforcement learning [10].

The RoboCup 2D community has also accumulated strong hand-coded client teams, base code, and client-side development frameworks. HELIOS teams have won the RoboCup 2D Simulation League multiple times [14]. HELIOS-base is a widely used open-source base team in the community [15, 16]. It provides rule-based player-client behaviors such as passing, shooting, dribbling, interception, positioning, and set plays, and these behaviors can serve as action abstractions for learning environments. To make RoboCup 2D client development easier, Pyrus reimplements the RCSS2D base-code interface in Python, targeting researchers who want to prototype player-client behavior and integrate machine-learning methods without working directly with C++ base teams [6]. The Cross Language Soccer Framework takes a different route toward easier client development by retaining the high-performance HELIOS base code while exposing a gRPC-based, language-agnostic client architecture that supports player-client development in languages such as C#, JavaScript, and Python [7]. Pyrus and the Cross Language Soccer Framework focus on the client side of the RCSS2D workflow, whereas R2D-RL targets the server/environment side that must organize state, rewards, action masks, reset control, and synchronization for MARL training.

2.2 Adapting RCSS2D for MARL Training

Multi-agent reinforcement learning (MARL) usually models an environment as a joint interaction process. At each environment step, the environment provides observations to multiple agents, the agents produce a joint action according to their policies, and the environment executes the joint action, transitions to the next state, and returns rewards, termination signals, and new observations. For football tasks, this formulation is particularly important because the match state is jointly driven by the simultaneous decisions of multiple players.

This formulation highlights the distinction between client-side simulator access and a training-time MARL environment. Client-side frameworks facilitate player-client implementation and simulator communication. However, client-side access alone does not define synchronized environment steps or expose the training-time semantics required by MARL algorithms, including centralized state, rewards, action masks, reset control, and termination signals. Therefore, adapting RCSS2D for MARL training requires an environment-side layer in addition to player-client development support.

Adapting RCSS2D to this MARL formulation first requires organizing the information needed for centralized training with decentralized execution (CTDE) [17, 18]. RCSS2D originally adopts a distributed competition structure in which each player is an independent client that sends commands to the server based on its own local perception. This structure is naturally aligned with decentralized execution. However, centralized training additionally requires the environment side to organize centralized state, local observations, joint actions, available actions, rewards, and termination signals. Thus, an RCSS2D-based MARL environment must preserve independent player-side execution while providing training-time global state and feedback.

The second issue is consistency between actions and state transitions. The original RCSS2D workflow is a real-time server loop: player clients must send commands to the server within the time window of each simulator cycle. If a command does not arrive within the corresponding cycle, the server still advances the match, and the command is not incorporated into the intended state transition. For reinforcement-learning training, the joint action submitted at an environment step must correspond to the subsequent transition, reward, and termination signal. Therefore, the MARL step, player commands, and server simulation cycle must be synchronized so that action execution and environment feedback belong to the same transition process.

Table 1: Comparison with representative football learning environments.

Environment	Scope	Simulator Base	Built-in AI	Action space	Action mask	Initial
GRF [2]	Full-field+scenario	Football Engine	Yes	High-level	No	Positi
NFootball [3]	Scenario	Python 2D	No	High-level	No	Track
Unity Soccer [19, 20]	Scenario	Unity	No	Low-level	No	Preser
Custom 2D Football [21]	Full-field	Custom 2D	No	Low-level	No	Positi
RoboCup 3D [22]	Full-field	SimSpark	No	Low-level	No	Defau
DM Control Soccer [23, 24]	Scenario	MuJoCo	No	Low-level	No	Rand
R2D-RL (ours)	Full-field+scenario	RCSS2D	Yes	High-level discrete + param. hybrid	Yes	Pos.-

The third issue is exposing the resulting process as a MARL environment interface. After training information and step synchronization are organized, the RCSS2D match workflow still needs to be wrapped as operations that learning frameworks can call, such as `reset()`, `step()`, observations, centralized state, available actions, rewards, and termination signals. This interface layer is what turns a competition-oriented server-client system into a training environment that can be used directly by modern MARL frameworks.

Figure 1 summarizes the RoboCup 2D ecosystem by taking RCSS2D as the reference point. RCSS2D [4, 5] provides the Full Match Competition server. From this full-match competition platform, Keepaway [8, 9] and HFO [10] derive lightweight Subtask MARL Environment settings. HELIOS-base [14], Pyrus [6], and CLSF [7] are RCSS2D-oriented client frameworks. With the environment-side layer described above, R2D-RL builds on the original RCSS2D server and modified HELIOS clients, and provides a Full Match MARL Environment.

2.3 Other Football Learning Environments

Beyond the RoboCup 2D ecosystem, football learning environments differ substantially in their research objectives and action-abstraction levels. Some are oriented toward football decision making and MARL benchmarks, where football is organized as a learning problem for algorithm evaluation, reward design, scenario-based learning, or large-scale training. Google Research Football (GRF) provides full-field play and Football Academy scenarios; its action set includes player-behavior actions such as movement, passing, shooting, sprinting, and dribbling, making it suitable for studying tactical coordination, reward design, curriculum tasks, and algorithmic benchmarking [2]. NFootball and the adaptive action-supervision framework of Fujii et al. connect football decision making with professional tracking data by using real-world multi-agent demonstrations and simulation-based tasks to study action supervision in reinforcement learning [3]. Unity ML-Agents soccer tasks, including Soccer Twos and Strikers vs. Goalie, use compact arenas, ray-cast observations, explicit rewards, and discrete branched actions that mainly correspond to movement and rotation, making them lightweight benchmarks for competitive and cooperative MARL [19, 20]. Recent simplified 11-vs-11 simulated robotic football work builds a custom 2D simulator and uses a simpler continuous action representation to improve training throughput for full-team MARL [21].

A second family is oriented toward robot soccer and embodied control, where the football interaction is coupled with physical movement, continuous control, or simulator protocols. RoboCup 3D simulation extends robot soccer to humanoid agents in a SimSpark-based three-dimensional simulator, requiring soccer decisions to be coordinated with locomotion, balance, and kicking skills [22]. DM Control Soccer builds multi-agent soccer tasks on MuJoCo-based articulated-body simulation and emphasizes continuous control, embodied interaction, and multi-agent coordination [23, 24].

Table 1 further contrasts R2D-RL with representative football learning environments. In the table, scope indicates whether an environment supports full-field games, isolated scenarios, or

R2D-RL Architecture and One-Step Synchronization

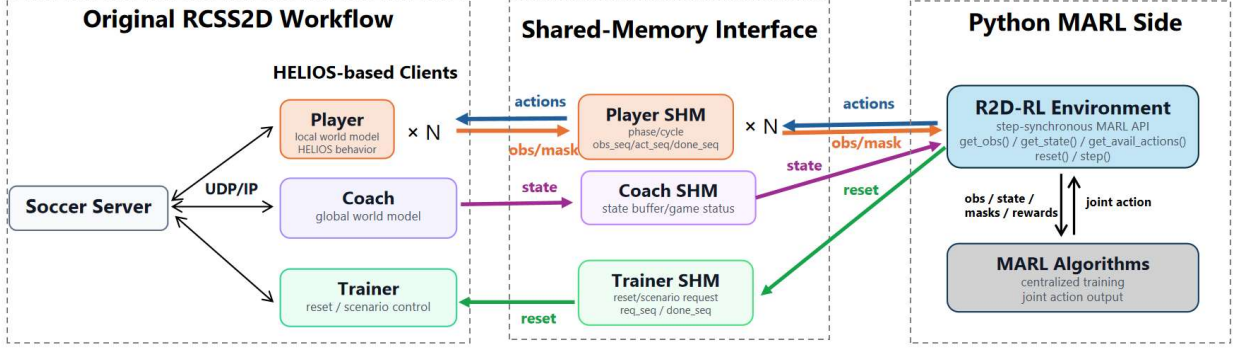


Figure 2: R2D-RL architecture and one-step synchronization. The original RCSS2D workflow with the soccer server and modified HELIOS-based player, coach, and trainer clients is preserved. Player, coach, and trainer shared-memory buffers exchange actions, observations and action masks, centralized state, and reset requests with the Python MARL environment, which provides step-synchronous interaction for MARL algorithms.

both; simulator base identifies the underlying engine or simulator; built-in AI indicates whether rule-based game or agent behavior is available without learning; action space distinguishes soccer-semantic high-level choices from lower-level physical movement or simulator-control commands; action mask indicates whether the environment exposes state-dependent available actions; and initial state describes the information used to start or restore an episode. Under these criteria, R2D-RL is the only environment that combines RCSS2D and HELIOS with both a high-level discrete action space and a parameterized hybrid action space, action masks, and scenario restoration through position, direction, and velocity.

3 Methodology

3.1 System Overview

R2D-RL is implemented as a Python MARL layer around the original RCSS2D server-client workflow. It runs the soccer server together with modified HELIOS player clients, coach, and trainer, while the Python environment communicates with these clients through shared memory and synchronizes them at the simulator-cycle level. By preserving the original soccer-server match logic and HELIOS behavior modules, R2D-RL converts the competition-oriented workflow into a step-based learning environment. Figure 2 summarizes this design.

3.2 Step-Synchronous Communication Protocol

R2D-RL uses a shared-memory sequence-counter protocol to organize the original RCSS2D server-client workflow as reinforcement-learning step(actions) calls. To focus learning on continuous tactical decision making, R2D-RL exposes policy decisions only during the play_on mode. Other play modes, such as kickoffs, free kicks, and referee-controlled restarts, remain handled by the RCSS2D rule flow and default HELIOS behaviors. Without changing the soccer server’s match logic, the protocol introduces a synchronization boundary between the modified HELIOS clients and the

Python environment: controlled players publish decision points, the environment writes actions for those decision points, and the players acknowledge that the actions have been consumed.

The player-side protocol is summarized in Table 2. Each controlled player maintains `phase`, `cycle`, `obs_seq`, `act_seq`, and `done_seq`. The `phase` field records the current play mode in the player’s world model, and `cycle` records the player-side simulator cycle. The three sequence counters respectively represent observation publication, action assignment, and action consumption. One synchronized environment step proceeds as follows.

Table 2: Player-side sequence fields.

Field	Meaning
<code>phase</code>	Current play mode.
<code>cycle</code>	Player-side simulator cycle.
<code>obs_seq</code>	Observation-publication sequence.
<code>act_seq</code>	Action-assignment sequence.
<code>done_seq</code>	Action-consumption sequence.

1. Each controlled player in the `play_on` phase writes its current observation and action mask to the player shared-memory buffer, then increments `obs_seq`.
2. The environment waits until the participating players satisfy `phase = play_on` and `obs_seq > act_seq`.
3. The environment reads the current player observations, action masks, and coach-side centralized state, and exposes them to the learning algorithm.
4. The learning algorithm returns a joint action for the controlled players.
5. The environment writes each selected action to the corresponding player buffer and sets `act_seq` to the target `obs_seq`.
6. Each player waits until `act_seq = obs_seq`, reads the assigned action, and invokes the corresponding HELIOS behavior or low-level command.
7. After consuming the action, each player updates `done_seq`. The environment waits until the participating players satisfy `done_seq = act_seq`, confirming that the joint action has been consumed.

This sequence-counter protocol explicitly separates observation publication, action assignment, and action consumption. As a result, in the `play_on` mode, each synchronized environment step is tied to one RCSS2D simulator-cycle decision point. A controlled player publishes its observation and action mask for that cycle, the environment writes the action for the corresponding `obs_seq`, and the player does not complete command dispatch until that action has been received. Together with RCSS2D’s native synchronization mode, which waits for player commands before advancing the server cycle, this barrier allows the coach-side centralized state published through shared memory to be aligned with the player-side observations at the same simulator cycle. The trainer client uses a `req_seq/done_seq` request-acknowledgment mechanism for reset and scenario initialization, bringing reset operations into the same synchronization design. Together, these mechanisms convert the original competition-oriented execution into repeatable step-synchronous MARL transitions.

3.3 RL Interface and Environment Semantics

This subsection defines the environment semantics exposed by R2D-RL to MARL algorithms, including play-mode handling, centralized state, local observations, action spaces, action masks, termination, rewards, opponent settings, and scenario initialization.

Play-mode handling. RCSS2D represents the current match state using play modes, including play_on, kickoff, free kick, kick in, corner kick, and goal kick modes. R2D-RL preserves this native play-mode mechanism. Because play_on corresponds to the continuous open-play phase in which regular movement, passing, shooting, and defensive decisions mainly occur, R2D-RL exposes policy decision frames only during play_on.

In restart modes such as kickoff, free kick, kick in, corner kick, and goal kick, the restart procedure is handled by the RCSS2D referee logic and HELIOS fallback behavior. During these non-play_on modes, the Python environment does not request policy actions or advance an environment step; the corresponding simulator cycles are not counted in the episode horizon. The environment enters the next environment step only after receiving the next synchronized play_on observations.

Therefore, the episode horizon in this paper is counted in Python environment steps. Each environment step corresponds one-to-one to a server cycle under the RCSS2D play_on mode.

State. The centralized state is supplied by the modified coach and is represented from the left-team perspective by default. In the current implementation, the coach state is a 136-dimensional vector, with dimension $4 + 22 \times 6 = 136$. The first four entries encode the ball x/y -position and x/y -velocity. The remaining entries correspond to 22 player slots, covering both teams. Each player slot contains six scalar features: x/y -position, x/y -velocity, body direction, and team identifier.

Observations. Local observations are generated by the modified players from their HELIOS world models. Each player observation is a flat 97-dimensional vector, with dimension $6 + 4 + 11 \times 4 + 10 \times 4 + 3 = 97$. The first six entries encode the player’s own x/y -position, x/y -velocity, stamina, and kickable flag. The next four entries encode the ball x/y -position and x/y -velocity. The following entries contain up to 11 opponent slots and up to 10 teammate slots, each with four scalar features: x/y -position and x/y -velocity. The final three entries are scalar indicators for game mode, side, and goalie status. Missing player slots are zero-padded, so the observation dimension remains fixed across full-field and small-sided settings.

Actions. The final execution layer in RCSS2D is formed by lower-level body commands. Under ordinary play_on control, the relevant body-command types include turn, dash, kick, catch, and tackle. On top of these lower-level body commands, R2D-RL defines two policy-level action spaces: Base and Hybrid.

The Hybrid action space directly exposes a lower-level body-command interface to the policy. It exposes turn, dash, kick, and catch; tackle is excluded because its unstable execution outcome can severely interfere with stable possession and dribbling control during training. In the R2D-RL Hybrid interface, each exposed body-command type is represented by a discrete command ID and corresponding normalized continuous arguments. The Hybrid action space also includes two special internal entries, HELIOS Fallback and empty, which are used only for internal control. Table 3 summarizes the current Hybrid actions and their corresponding parameterization.

The Base action space is a HELIOS-based high-level semantic interface. The policy selects only a soccer-semantic action label; the modified HELIOS client then invokes the corresponding behavior module. This module determines the actual lower-level body command or command sequence and

Table 3: Hybrid action parameterization in R2D-RL.

Action	ID	Parameterization
turn	0	angle $\theta \in [-180^\circ, 180^\circ]$
dash	1	power $p \in [1, 100]$
kick	2	power $p \in [0, 100]$, angle $\theta \in [-180^\circ, 180^\circ]$
catch	3	angle $\theta \in [-90^\circ, 90^\circ]$
HELIOS Fallback	4	no continuous parameter
empty	5	no continuous parameter

Table 4: High-level discrete action set in R2D-RL.

Actions		
Tackle	Intercept	Shoot
Advance	Direct Pass	Lead Pass
Through Pass	Hold Ball	Catch
Dribble Up	Dribble Down	Dribble Left
Dribble Right	Move Up	Move Down
Move Left	Move Right	HELIOS Fallback
Empty		

computes the required continuous arguments internally from the current world model. Tackle is retained in Base because HELIOS constrains its executability and execution, unlike directly parameterized Hybrid commands. The direction-labeled Move and Dribble actions are R2D-RL-defined directional primitives expressed as field-coordinate offsets under the left-team field convention, rather than as agent body-relative directions. Move actions denote short off-ball movements with a step size of 3.0 along the field x - or y -axis from the player’s current position, while Dribble actions use the same directional offsets when the player is kickable and execute ball-carrying dribble behavior. For pass actions, the policy selects only the pass type, while the corresponding HELIOS pass behavior selects the receiver and target point internally using distance-based feasibility checks from the current world model. The current Base actions are summarized in Table 4. In the reported experiments, the **advance** behavior is disabled because it can immediately move the ball over a long distance and introduce large discontinuous changes in possession and field position.

Action masks. For the Base action space, R2D-RL exposes state-dependent action masks derived from the executability checks in the HELIOS behavior modules. These masks restrict unavailable soccer behaviors at each decision point. For example, shooting, passing, or catching actions are enabled only when their corresponding HELIOS-side conditions are satisfied. Shooting is treated as a priority action: when the shooting condition is satisfied, all other Base actions are masked out. In the reported experiments, HELIOS Fallback and empty actions are masked out for policy selection and are used only for internal control. For the Hybrid action space, command-level masks restrict low-level commands using their simulator-side executability conditions: kick commands are enabled only for kickable players, and catch commands are enabled only for goalies satisfying the catch-executability conditions. The HELIOS Fallback and empty entries are also masked out for policy selection and are used only for internal control. Detailed Base and Hybrid action-mask conditions are summarized in Appendix A, Tables 7 and 8.

Termination. R2D-RL implements configurable episode-termination conditions. **Goal event:** the controlled team scores or concedes a goal. **Out-of-play event:** the ball goes out of play.

Possession loss: the defending side is assigned ball ownership by the environment’s nearest-player possession rule. This requires a defender to be within the kickable-distance threshold of the ball, with no controlled-team player equally close or closer to the ball. **Timeout:** the episode reaches the configured horizon. Timeout is counted by Python environment steps rather than raw RCSS2D simulator cycles; only synchronized play_on environment steps advance the timeout counter.

Rewards. R2D-RL supports a team-level scoring reward and an optional MaxEPV shaping reward. The scoring reward is returned as a shared team reward for all controlled agents: it is +1 when the controlled team scores and −1 when it concedes a goal. Because relying only on goal events makes the reward signal sparse, we design a MaxEPV shaping reward following [25].

Specifically, MaxEPV uses a precomputed 32×50 expected possession value (EPV) grid over the pitch [26]. Each grid value represents the expected value that the current possession or attack will eventually be converted into a goal when the ball is located in the corresponding pitch region; higher values indicate more favorable attacking locations. The EPV grid used in our experiments contains normalized values ranging from 0.0041 to 0.5714. Let v_t denote the EPV value of the ball location at time t , and let m_t denote the maximum EPV value reached so far in the current episode. To avoid rewarding passive ball movement caused by opponents or uncontrolled dynamics, we update m_t only when at least one controlled-team player is kickable. We define $k_t \in \{0, 1\}$ as the indicator of this condition:

$$m_t = \begin{cases} \max(m_{t-1}, v_t), & k_t = 1, \\ m_{t-1}, & k_t = 0. \end{cases}$$

The EPV shaping reward is defined as the positive increase in this running maximum,

$$r_t^{\text{EPV}} = \lambda \max(0, m_t - m_{t-1}),$$

where $\lambda = 2$ in our experiments. When the controlled team scores, R2D-RL additionally adds a goal-completion MaxEPV reward equal to $\lambda(v_{\max} - m_t)$, where $v_{\max}$ is the maximum value in the EPV grid. The final reward is the sum of the scoring reward and the optional MaxEPV shaping reward.

In all experiments, possession-loss, out-of-play, and timeout terminations do not add any extra reward or penalty.

Built-in AI. Opponent and non-controlled teammate behaviors are handled through the same action interface as controlled players. In the reported benchmarks, the environment assigns the HELIOS Fallback action to all opponent players and non-controlled teammates at each step, while the learning policy selects actions only for the controlled players. This provides built-in AI control for background players and fixed rule-based opponents for baseline training and evaluation.

Scenario reset. R2D-RL supports both built-in benchmark start states and user-defined scenario initialization. The built-in start states are used for the reported front-goal and full-field experiments, while custom scenarios can specify the initial ball state and player states, including position, body direction, and velocity. At reset(), the Python environment selects a built-in or custom start state and sends it to the trainer through shared memory. Instead of restarting the soccer server for every episode, which is too slow for training-scale rollout collection, the trainer resets the running server state by directly moving the ball and players to the requested states. After repositioning, the environment waits for several warm-up simulator cycles so that player world models and observations are updated consistently. Once the trainer acknowledgement and synchronized play_on

observations are available, the environment clears episode-level bookkeeping and returns the first observation for rollout.

4 Experiments and Results

The experiments evaluate R2D-RL from three complementary perspectives: parallel sampling throughput, progressive front-goal scenarios, and full-field 11-vs-11 benchmarks.

4.1 Parallel Sampling Throughput

In practical reinforcement-learning training, collecting rollout data with parallel environment instances is a common way to improve sampling efficiency and accelerate training. We therefore evaluate how environment-step throughput scales with the number of parallel environment instances. Here, one environment step denotes one joint action decision step at the R2D-RL environment API level: all controlled agents submit one joint action, and the environment returns the next observation, reward, and termination signal.

The experiment is conducted on a compute node equipped with two AMD EPYC 7302 16-Core processors. The node has two sockets, 16 physical cores per socket, and two hardware threads per core, for a total of 64 logical CPUs. A total of 32 logical CPUs are allocated through SLURM, and throughput is reported as environment steps per day. The benchmark interacts with R2D-RL using random legal actions and does not include policy inference or learning updates.

Figure 3 reports 11-vs-11 full-field environment-step throughput under two action-space configurations: the Base action space and the Hybrid action space. Throughput increases with the number of parallel environment instances in both settings and reaches its maximum at 14 parallel instances. The Base action space reaches 34.85 million environment steps per day, while the Hybrid action space reaches 41.65 million environment steps per day. For larger numbers of parallel environment instances, the measured throughput remains below these peak values.

4.2 Algorithms

R2D-RL provides two action spaces: the Base action space and the Hybrid action space. To evaluate these two action spaces, we use multi-agent proximal policy optimization (MAPPO) [27] and QMIX [18] for the Base action space. For the Hybrid action space, we use an independent ParaDQN adapted from the single-agent parameterized-action formulation [28]. ParaDQN is used as a parameter-sharing independent learner: all controlled agents share one Q network and one parameter actor, while each agent selects its own parameterized Hybrid action from its local observation. The parameter actor outputs the continuous action parameters, and the Q network selects the discrete Hybrid command conditioned on those parameters. No centralized state input, centralized critic, or value decomposition is used. This design is included to validate the trainability of the Hybrid action interface rather than to serve as a fully centralized MARL method. Detailed algorithm training settings are provided in Appendix B, Tables 9, 10, and 11.

4.3 Front-Goal Scenarios

Before evaluating full-field match performance, we introduce five controlled front-goal scenarios that provide a progression of possession-based attacking tasks. These scenarios share the same attacking direction toward the right goal and are used as controlled settings before moving to full-field matches. They are organized as a progressive difficulty ladder, from single-attacker finishing

11-vs-11 Action-Space Throughput under 32 Logical CPUs

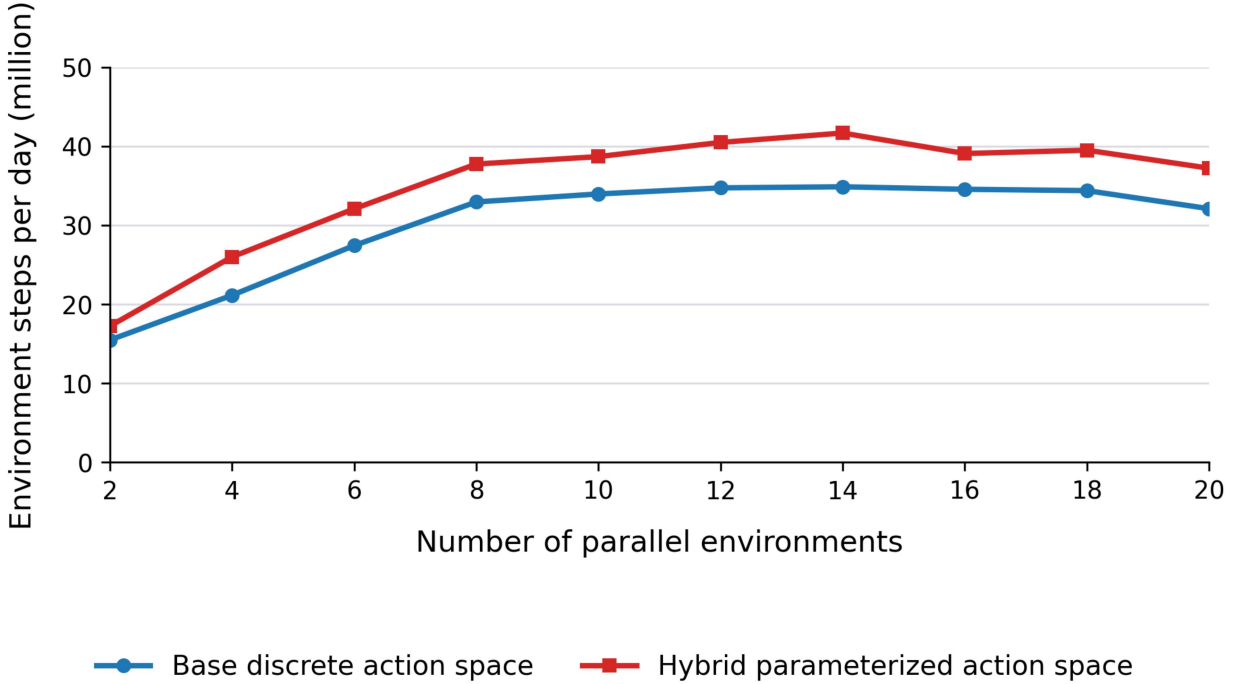


Figure 3: Parallel sampling throughput of R2D-RL in 11-vs-11 full matches with Base and Hybrid action spaces. Throughput is reported as environment steps per day under a 32-logical-CPU SLURM allocation.

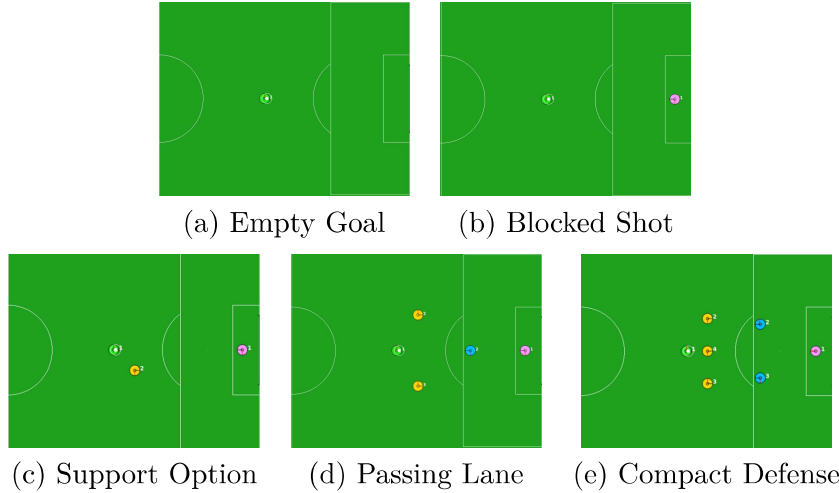


Figure 4: Initial frames of the front-goal scenarios. Yellow and green players denote attacking players, with the green player carrying the ball. Blue players denote defending players, and the pink player denotes the goalkeeper.

to multi-attacker coordination under increasing defensive pressure, allowing us to examine whether task performance decreases as coordination demands increase and whether MaxEPV shaping and

action masks improve learning in harder settings.

As shown by the initial frames in Fig. 4(a)–(e), the five scenarios are Empty Goal (a), Blocked Shot (b), Support Option (c), Passing Lane (d), and Compact Defense (e). Starting from the empty-goal finishing task, they progressively increase task complexity by varying the number of active attackers and defenders, teammate options, passing-lane structures, and defensive pressure. Table 5 summarizes the main design focus of each scenario. The attacking players are controlled by the learning policy, while the defending players and goalkeeper are assigned the HELIOS Fallback action as built-in AI behavior.

Table 5: Design purposes of front-goal scenarios.

Scenario	Main focus
Empty Goal	Ball progression and finishing
Blocked Shot	Pressured finishing
Support Option	Teammate support
Passing Lane	Off-ball support
Compact Defense	Coordinated attack under compact defense

4.4 Single-Attacker Front-Goal Scenarios

We first evaluate the basic trainability of R2D-RL in two single-attacker front-goal scenarios: Empty Goal and Blocked Shot. Empty Goal is a 1-vs-0 finishing setting, while Blocked Shot adds direct defensive pressure as a 1-vs-1 setting. These scenarios are designed to verify that the environment interface, action execution, reset mechanism, and evaluation pipeline can support stable reinforcement-learning training. They also test whether both the Base discrete action space and the Hybrid parameterized action space are trainable in simple finishing and pressured finishing settings, and whether MaxEPV shaping and action masks affect learning behavior in these settings.

Each episode starts with the attacking side in possession. Empty Goal has a maximum horizon of 200 environment steps, and Blocked Shot has a maximum horizon of 300 environment steps. The enabled episode-termination conditions are goal event, out-of-play event, possession loss, and timeout. We evaluate four environment-component settings: MaxEPV+mask, mask only, MaxEPV only, and no EPV/mask. All settings use the team-level scoring reward, while MaxEPV shaping and action masks are enabled according to the selected setting. Both scenarios are trained for 3M environment steps with three random seeds. Although these scenarios contain only one controlled attacker, we still use MAPPO and QMIX for the Base action space to keep the algorithm interface, action space, training procedure, and evaluation protocol consistent with the later multi-attacker and full-field benchmark. For the Hybrid action space, we use the ParaDQN setting described above.

Figure 5 and Table 12 in Appendix C report the goal rates for Empty Goal and Blocked Shot. In Empty Goal, ParaDQN reaches 1.00 ± 0.00 in all four component settings; QMIX reaches 1.00 ± 0.00 in the two mask-enabled settings and 0.67 ± 0.47 in the two mask-disabled settings; MAPPO ranges from 0.94 ± 0.08 to 1.00 ± 0.00 . In Blocked Shot, QMIX records 0.88 ± 0.02 and 0.87 ± 0.07 in the two mask-enabled settings and 0.00 ± 0.00 in the two mask-disabled settings. MAPPO records 0.66 ± 0.03 , 0.69 ± 0.08 , 0.89 ± 0.02 , and 0.86 ± 0.05 across the four settings. ParaDQN obtains its highest Blocked Shot result, 0.28 ± 0.07 , when both MaxEPV and masks are enabled. Appendix Fig. 8 provides qualitative QMIX examples for the mask-off and mask-on Blocked Shot cases.

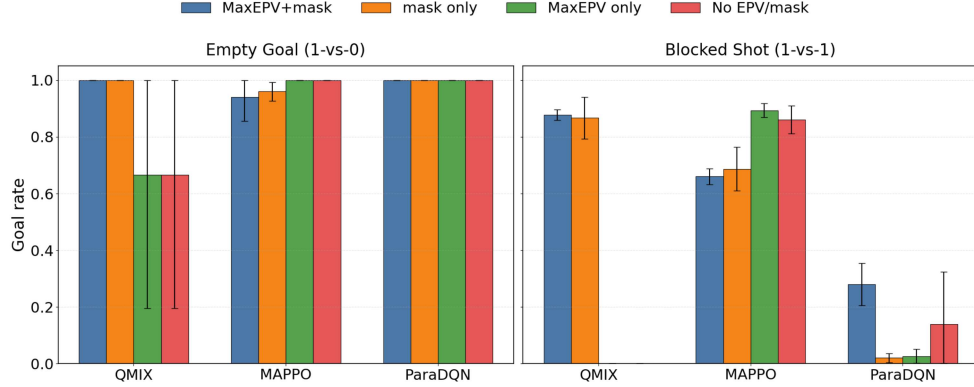


Figure 5: Single-attacker front-goal results. Bars show the goal rate over evaluation episodes, summarized as mean and standard deviation over three random seeds. Detailed numerical values are reported in Appendix Table 12.

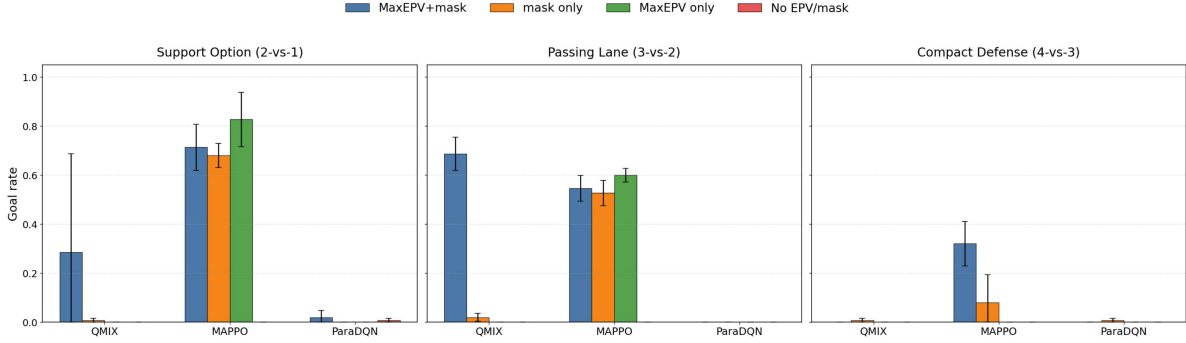


Figure 6: Multi-attacker front-goal results. Bars show the goal rate over evaluation episodes, summarized as mean and standard deviation over three random seeds. Colors denote the four component settings: MaxEPV+mask, mask only, MaxEPV only, and no EPV/mask. Detailed numerical values are reported in Appendix Table 12.

4.5 Multi-Attacker Front-Goal Scenarios

Support Option, Passing Lane, and Compact Defense extend the front-goal evaluation to multiple controlled attackers. These scenarios use the same possession-based front-goal setup as the single-attacker settings: the controlled attacking side starts in possession toward the right goal, and episodes terminate on a Goal event, an Out-of-play event, Possession loss, or Timeout as defined in Section 3. Compared with the single-attacker settings, these scenarios require teammate support, passing-lane selection, off-ball positioning, and multi-agent credit assignment. Each episode has a maximum horizon of 300 environment steps. We evaluate the same four environment-component settings as in the single-attacker experiments. All settings use the team-level scoring reward, while MaxEPV shaping and action masks are enabled according to the selected setting. Each method is trained for 5M environment steps with three random seeds.

We report the goal rate over fixed evaluation episodes as the primary evaluation metric, with results summarized as mean and standard deviation over three random seeds.

Figure 6 and Table 12 in Appendix C report the goal rates for Support Option, Passing Lane, and Compact Defense. The highest goal rate in each scenario is 0.83 ± 0.11 for Support Option, 0.69 ± 0.07 for Passing Lane, and 0.32 ± 0.09 for Compact Defense. MAPPO records 0.83 ± 0.11

in Support Option, 0.60 ± 0.03 in Passing Lane, and 0.32 ± 0.09 in Compact Defense. QMIX records 0.69 ± 0.07 in Passing Lane with MaxEPV+mask, 0.28 ± 0.40 in Support Option with MaxEPV+mask, and near-zero values in Compact Defense. ParaDQN remains near zero across the multi-attacker scenarios. In the no-EPV/mask setting, nearly all multi-attacker combinations have zero goal rate. Appendix Fig. 9 provides a qualitative Compact Defense example from the MAPPO MaxEPV+mask setting.

4.6 Full-Field Benchmark

We further evaluate R2D-RL under full-field match dynamics in the standard 11-vs-11 RCSS2D setting. Each episode has a maximum horizon of 3000 environment steps. In the reported full-field experiments, the enabled episode-termination condition is timeout; goal events trigger native RCSS2D goal restarts but do not terminate the episode. The learning policy controls all players on the left team, while opponent players are assigned the HELIOS Fallback action as built-in AI behavior. To reduce fixed kickoff bias, the default kickoff side is alternated across timeout resets, while goal restarts follow the native RCSS2D rule in which the conceding side kicks off.

The goal of this experiment is to evaluate whether R2D-RL, as an environment and benchmark, can support end-to-end training, synchronized sampling, and standardized evaluation under complete RCSS2D match dynamics, while providing a reproducible evaluation setting for future large-scale algorithmic studies. Full 11-vs-11 football typically requires longer training budgets, curriculum learning, self-play, demonstrations, pretraining, or algorithm-specific design.

All full-field experiments use the team-level scoring reward, MaxEPV shaping, and action masks. They are conducted with three random seeds and trained for 30M environment steps. We use goal difference as the primary evaluation metric and report `max_epv_improvement` as an auxiliary metric for ball progression toward higher-value attacking regions. Figure 7 shows a left-team kickoff example for the full-field benchmark.

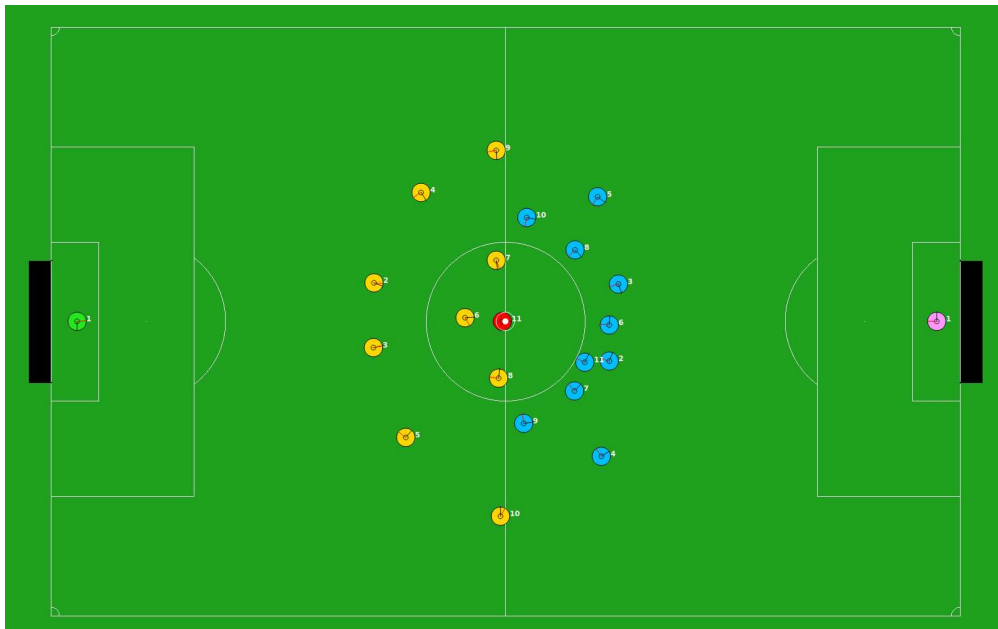


Figure 7: Left-team kickoff example for the 11-vs-11 full-field benchmark in R2D-RL.

Table 6: 11-vs-11 full-field benchmark results at 30M environment steps. Values are mean and standard deviation over three completed seeds.

Algorithm	Goal diff.	max_epv_improvement
MAPPO @30M	-10.61 ± 1.64	0.0317 ± 0.0235
QMIX @30M	-19.00 ± 8.60	0.0323 ± 0.0355
ParaDQN @30M	-25.90 ± 2.69	0.0014 ± 0.0002

Table 6 summarizes the final 11-vs-11 full-field benchmark results for the policies trained for 30M environment steps. For each random seed, the final trained policy is evaluated over 50 episodes. Goal difference is computed per episode as the number of goals scored by the controlled left team minus the number of goals scored by the opponent team, and the table reports the mean and standard deviation over the three seed-level averages. The auxiliary metric max_epv_improvement is computed per episode as the maximum MaxEPV value reached during the episode minus the initial EPV value assigned when the controlled team first becomes kickable at kickoff, and is aggregated in the same way. Additional full-field results in Appendix D include the corresponding return curves in Fig. 10 and qualitative rollout examples in Figs. 11, 12, and 13. At 30M environment steps, MAPPO obtains a goal difference of -10.61 ± 1.64 , QMIX obtains -19.00 ± 8.60 , and ParaDQN obtains -25.90 ± 2.69 . The max_epv_improvement values are 0.0317 ± 0.0235 for MAPPO, 0.0323 ± 0.0355 for QMIX, and 0.0014 ± 0.0002 for ParaDQN.

4.7 Discussion

The experiments show that R2D-RL supports a progressive football MARL benchmark. The throughput results demonstrate that the environment can be sampled in parallel at practical scale. Beyond 14 parallel instances, throughput no longer improves substantially, indicating that the allocated CPU resources become saturated and that simulator-client synchronization, inter-process communication, and reset handling begin to limit parallel scaling.

The front-goal scenarios provide controlled short-horizon attacking tasks and reveal the increasing difficulty from individual finishing to pressured and multi-player attacking decisions. In the Empty Goal task, all three algorithms achieve goal rates close to one under most component settings, indicating that the R2D-RL interface and evaluation pipeline support stable learning for QMIX, MAPPO, and ParaDQN in a simple finishing scenario. In the Blocked Shot task, the lower goal rates compared with Empty Goal indicate that the HELIOS Fallback defender introduces meaningful defensive pressure.

The single-attacker results further show that the algorithms differ in their dependence on action masks. For QMIX with the Base action space, action masks are critical in the pressured 1-vs-1 setting. Qualitative examples in Appendix Fig. 8 are consistent with this pattern: without masks, the policy is not forced to switch to the shooting action when the shooting condition is satisfied, and may continue selecting dribbling or forward-movement actions until the ball enters the goalkeeper’s catchable area. The shoot-priority mask reduces this exploration burden by removing competing actions whenever shooting is executable, so the learner does not need to discover the switching timing from sparse outcomes alone. MAPPO is more robust in this single-attacker setting, while ParaDQN with the Hybrid action space must jointly learn a discrete command type and continuous action parameters.

The multi-attacker results show a clear difficulty gradient across Support Option, Passing Lane, and Compact Defense. This trend is consistent with the scenario design: the tasks progress from using a supporting teammate to selecting passing lanes and attacking against compact defense. MAPPO provides the most stable learning results across the three multi-attacker scenarios. QMIX can learn selected settings, but it is more sensitive to random seeds. ParaDQN remains near

zero across the multi-attacker scenarios, indicating that the Hybrid parameterized-action interface becomes substantially harder in complex tasks. The multi-attacker results also show that MaxEPV shaping and action masks are effective in complex settings. Nearly all no-EPV/mask settings fail to score, and in the Compact Defense scenario only MAPPO obtains a clearly nonzero result when both MaxEPV shaping and action masks are enabled; Appendix Fig. 9 provides a qualitative example from this setting. In addition, QMIX’s dependence on action masks and MaxEPV reward in Support Option and Passing Lane further indicates that these components are useful in complex multi-attacker settings.

Across the 11-vs-11 baselines, the relative algorithm behavior is consistent with the trends observed in the front-goal scenarios: MAPPO is the most stable, QMIX shows stronger seed-level variation, and ParaDQN remains harder to train in the more complex setting. The `max_epv_improvement` values provide a complementary view of attacking progress. MAPPO and QMIX reach similar average values, but MAPPO is more stable. Evaluation sequences further show that MAPPO and some QMIX seeds can produce early forms of multi-agent coordination and attacking progression, as illustrated in Appendix Figs. 11 and 12. At the same time, the qualitative rollouts reveal several limitations, including fixed-direction off-ball movement far from the ball and the lack of clear penalty-area-specific goalkeeper behavior; Appendix Fig. 13 illustrates the off-ball movement limitation. ParaDQN remains close to the initial EPV baseline, and qualitative inspection of ParaDQN rollouts also shows that controlled players often remain nearly stationary and turn in place, rather than producing sustained ball progression.

5 Limitations and Future Work

R2D-RL currently evaluates learned policies mainly against fixed built-in AI opponents. In the reported benchmarks, the controlled side is selected for learning, such as the full left team in full-field tasks or all designated attackers in front-goal scenarios, while the opposing players are assigned the HELIOS Fallback action. This setting provides stable rule-based opponents and makes baseline training and evaluation reproducible. However, it also limits the diversity of tactical behaviors observed during training. Future work will extend the benchmark to self-play, learned opponents, and opponent pools with multiple built-in or learned policies, enabling evaluation of robustness and generalization against more diverse opponent behaviors.

The reported baselines are also limited by finite training budgets in both the scenario tasks and the 11-vs-11 full-field benchmark. These experiments are intended to validate R2D-RL as an environment and benchmark across progressively harder football tasks, rather than to exhaust the final performance of each learning algorithm. Longer training horizons, curriculum learning, self-play, demonstrations, pretraining, or algorithm-specific designs may substantially improve policy quality and should be investigated in future work.

R2D-RL also relies on HELIOS-derived high-level actions and action masks in the Base action space. Although the experimental results indicate that this design improves learning efficiency, both the high-level action definitions and the mask decisions are shaped by hand-coded HELIOS/R2D-RL rules, and therefore introduce inductive biases. Future work will investigate less hand-coded action abstractions and alternative mask designs to reduce this dependence.

Another limitation is that learning decisions are exposed only during `play_on`. This design keeps MARL transitions aligned with continuous open-play interactions and avoids mixing them with discrete referee-controlled restart procedures. However, restart and set-piece situations, such as kickoffs, free kicks, goal kicks, and penalty kicks, are also important components of full soccer strategy. In the current version, these situations are handled by the native RCSS2D rule flow and

HELIOS fallback behavior rather than by learned policies. Future work will extend R2D-RL to support learning in selected non-play_on modes, allowing policies to learn set-piece and restart behaviors while preserving the synchronization guarantees of the current environment.

6 Conclusion

This paper presented R2D-RL, a cycle-synchronized MARL environment that bridges the original RCSS2D simulator, HELIOS-based player clients, coach and trainer processes, and Python learning algorithms. Rather than replacing the RoboCup 2D ecosystem with a simplified simulator, R2D-RL preserves the original server-client structure and exposes it through training-oriented environment semantics, including synchronized observations, centralized state, policy-level action spaces, action masks, rewards, termination conditions, and scenario reset. This design positions R2D-RL as a full-match MARL environment built on RCSS2D while remaining compatible with the established RoboCup 2D workflow.

The benchmark results show that R2D-RL supports progressive football MARL experiments from controlled front-goal scenarios to full 11-vs-11 match dynamics. The environment can be sampled in parallel at practical scale, the single-attacker scenarios verify the basic trainability of both Base and Hybrid action spaces, and the multi-attacker and full-field settings expose increasing difficulty in teammate use, passing-lane selection, credit assignment, and long-horizon attacking progression. The results also show that MaxEPV shaping and action masks are useful environment components for complex football tasks, while the remaining gaps in full-field behavior highlight the need for larger-scale training, richer curricula, self-play, and less hand-coded action abstractions. Overall, R2D-RL provides a reproducible foundation for studying learning-based decision making in RoboCup 2D under realistic full-match dynamics.

References

- [1] Keisuke Fujii. *Machine Learning in Sports: Open Approach for Next Play Analytics*. Springer-Briefs in Computer Science. Springer Singapore, 2025. doi: 10.1007/978-981-96-1445-5. URL <https://doi.org/10.1007/978-981-96-1445-5>.
- [2] Karol Kurach, Anton Raichuk, Piotr Stańczyk, Michał Zajac, Olivier Bachem, Lasse Espeholt, Carlos Riquelme, Damien Vincent, Marcin Michalski, Olivier Bousquet, and Sylvain Gelly. Google Research Football: A novel reinforcement learning environment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 4501–4510, 2020. doi: 10.1609/aaai.v34i04.5878.
- [3] Keisuke Fujii, Kazushi Tsutsui, Atom Scott, Hiroshi Nakahara, Naoya Takeishi, and Yoshinobu Kawahara. Adaptive action supervision in reinforcement learning from real-world multi-agent demonstrations. In *Proceedings of the 16th International Conference on Agents and Artificial Intelligence (ICAART 2024)*, volume 2, pages 27–39, 2024. doi: 10.5220/0012261100003636. URL <https://arxiv.org/abs/2305.13030>.
- [4] Itsuki Noda, Hitoshi Matsubara, Kazuo Hiraki, and Ian Frank. Soccer server: A tool for research on multiagent systems. *Applied Artificial Intelligence*, 12(2–3):233–250, 1998. doi: 10.1080/088395198117059.
- [5] RoboCup Soccer Simulator Maintenance Group. RoboCup Soccer Simulator Server. <https://github.com/rcsoccersim/rcssserver>, 2026. Accessed: 2026-05-08.

- [6] Nader Zare, Aref Sayareh, Omid Amini, Mahtab Sarvmaili, Arad Firouzkouhi, Stan Matwin, and Amilcar Soares. Pyrus base: An open source python framework for the RoboCup 2d soccer simulation. *arXiv preprint arXiv:2307.16875*, 2023. doi: 10.48550/arXiv.2307.16875.
- [7] Nader Zare, Aref Sayareh, Alireza Sadraei, Arad Firouzkouhi, and Amilcar Soares. Cross language soccer framework: An open source framework for the RoboCup 2d soccer simulation. *arXiv preprint arXiv:2406.05621*, 2024. doi: 10.48550/arXiv.2406.05621.
- [8] Peter Stone, Richard S. Sutton, and Gregory Kuhlmann. Reinforcement learning for RoboCup soccer keepaway. *Adaptive Behavior*, 13(3):165–188, 2005. doi: 10.1177/105971230501300301.
- [9] Peter Stone, Gregory Kuhlmann, Matthew E. Taylor, and Yaxin Liu. Keepaway soccer: From machine learning testbed to benchmark. In Itsuki Noda, Adam Jacoff, Ansgar Bredendfeld, and Yasutake Takahashi, editors, *RoboCup 2005: Robot Soccer World Cup IX*, volume 4020 of *Lecture Notes in Computer Science*, pages 93–105. Springer, 2006. doi: 10.1007/11780519_9.
- [10] Shivaram Kalyanakrishnan, Yaxin Liu, and Peter Stone. Half field offense in RoboCup soccer: A multiagent reinforcement learning case study. In Gerhard Lakemeyer, Elizabeth Sklar, Domenico Sorrenti, and Tomoichi Takahashi, editors, *RoboCup 2006: Robot Soccer World Cup X*, volume 4434 of *Lecture Notes in Computer Science*, pages 72–85. Springer, 2007. doi: 10.1007/978-3-540-74024-7_7.
- [11] Hiroaki Kitano, Milind Tambe, Peter Stone, Manuela Veloso, Silvia Coradeschi, Eiichi Osawa, Hitoshi Matsubara, Itsuki Noda, and Minoru Asada. The RoboCup synthetic agent challenge 97. In *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence*, pages 24–29, San Francisco, CA, 1997. Morgan Kaufmann.
- [12] RoboCup Federation. RoboCupSoccer Simulation League. <https://www.robocup.org/leagues/23>, 2026. Accessed: 2026-05-10.
- [13] RoboCup Federation. RoboCupSoccer Simulation 2D League. <https://www.robocup.org/leagues/24>, 2026. Accessed: 2026-05-10.
- [14] Hidehisa Akiyama and Tomoharu Nakashima. HELIOS2012: Robocup 2012 soccer simulation 2d league champion. In *RoboCup 2012: Robot Soccer World Cup XVI*, volume 7500 of *Lecture Notes in Computer Science*, pages 13–19. Springer, 2013. doi: 10.1007/978-3-642-39250-4_2.
- [15] Hidehisa Akiyama and Tomoharu Nakashima. HELIOS base: An open source package for the RoboCup soccer 2d simulation. In *RoboCup 2013: Robot World Cup XVII*, Lecture Notes in Computer Science, pages 528–535. Springer, 2014. doi: 10.1007/978-3-662-44468-9_46.
- [16] HELIOS-base. <https://github.com/helios-base/helios-base>. Accessed: 2026-06-13.
- [17] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [18] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4295–4304. PMLR, 2018.

- [19] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2018. doi: 10.48550/arXiv.1809.02627.
- [20] Unity Technologies. Unity ML-Agents Toolkit: Example learning environments. <https://unity-technologies.github.io/ml-agents/Learning-Environment-Examples/>, 2026. Accessed: 2026-05-10.
- [21] Andries Smit, Herman A. Engelbrecht, Willie Brink, and Arnu Pretorius. Scaling multi-agent reinforcement learning to full 11 versus 11 simulated robotic football. *Autonomous Agents and Multi-Agent Systems*, 37(1):20, 2023. doi: 10.1007/s10458-023-09603-y.
- [22] RoboCupSoccer Simulation League. RoboCup 3D Soccer Simulation League. <https://ssim.robocup.org/3d-simulation/>, 2026. Accessed: 2026-05-22.
- [23] Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm_control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020. doi: 10.1016/j.simpa.2020.100022.
- [24] Farama Foundation. Shimmy: DM Control multi-agent soccer documentation. https://shimmy.farama.org/environments/dm_multi/, 2026. Accessed: 2026-05-10.
- [25] Hiroshi Nakahara, Kazushi Tsutsui, Kazuya Takeda, and Keisuke Fujii. Action valuation of on- and off-ball soccer players based on multi-agent deep reinforcement learning. *IEEE Access*, 11:131237–131244, 2023. doi: 10.1109/ACCESS.2023.3336425.
- [26] Friends of Tracking Data. LaurieOnTracking: EPV data. <https://github.com/Friends-of-Tracking-Data-FoTD/LaurieOnTracking>. Accessed: 2026-06-09.
- [27] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, volume 35, pages 24611–24624, 2022.
- [28] Jiechao Xiong, Qing Wang, Zhuoran Yang, Peng Sun, Lei Han, Yang Zheng, Haobo Fu, Tong Zhang, Ji Liu, and Han Liu. Parametrized deep q-networks learning: Reinforcement learning with discrete-continuous hybrid action space. *arXiv preprint arXiv:1810.06394*, 2018. doi: 10.48550/arXiv.1810.06394.

A Action Mask Details

This appendix summarizes the action-mask logic used by R2D-RL. Action masks are generated by the modified HELIOS player client at each synchronized play_on decision frame and are passed to the Python environment through shared memory. A mask entry of 1 means that the corresponding policy-level action is currently selectable, while a mask entry of 0 means that the action is unavailable for policy selection.

For the Base action space, the mask is a 19-dimensional binary vector for each controlled player. The enabled conditions are summarized in Table 7. If the shooting action is enabled, it is given priority and all other Base actions are masked out. When dynamic action masking is disabled, R2D-RL uses a static Base mask in which the normally selectable actions remain available, while advance, HELIOS Fallback, and empty remain masked out.

Table 7: Base action-mask conditions in R2D-RL.

ID	Action	Enabled condition
0	tackle	HELIOS basic tackle executable; execution constrained by HELIOS.
1	shoot	Kickable; not indirect free kick; not stopped; strict shoot check executable. If enabled, all other Base actions are masked out.
2	intercept	Not kickable; no kickable teammate; interception logic selects this player.
3	advance	Disabled in the reported experiments.
4	direct pass	Kickable; direct-pass candidate; one-step kick executable.
5	lead pass	Kickable; lead-pass candidate; one-step kick executable.
6	through pass	Kickable; through-pass candidate; one-step kick executable.
7	hold	HELIOS hold-ball behavior executable.
8	catch	Goalie; not frozen; in own penalty area; valid play mode and ball position; catch ban ended; ball within catchable area minus 0.05.
9	dribble up	Kickable.
10	dribble down	Kickable.
11	dribble left	Kickable.
12	dribble right	Kickable.
13	move up	Not kickable; 3.0-m left-team field offset remains in pitch.
14	move down	Not kickable; 3.0-m left-team field offset remains in pitch.
15	move left	Not kickable; 3.0-m left-team field offset remains in pitch.
16	move right	Not kickable; 3.0-m left-team field offset remains in pitch.
17	HELIOS Fallback	Disabled for policy selection; internal control only.
18	empty	Disabled for policy selection; internal control only.

For the Hybrid action space, the mask is a 6-dimensional binary vector for each controlled player. The Hybrid mask controls which lower-level command type can be selected by the policy, while the continuous command arguments are supplied separately by the parameter heads. Table 8 summarizes the current Hybrid mask.

Table 8: Hybrid action-mask conditions in R2D-RL.

ID	Action	Enabled condition
0	turn	The player is not in the unfrozen kickable-command branch; equivalently, the player is not kickable or is frozen.
1	dash	The player is not kickable and is not frozen.
2	kick	The player is kickable and is not frozen.
3	catch	The same catch-executability condition as in the Base action space is satisfied.
4	HELIOS Fallback	Disabled for policy selection and reserved for internal control.
5	empty	Disabled for policy selection and reserved for internal control.

Thus, in ordinary non-frozen frames, the Hybrid mask separates on-ball command selection, where kick and executable catch actions can be selected, from off-ball command selection, where turn, dash, and executable catch actions can be selected. HELIOS Fallback and empty are kept as internal entries rather than policy-selectable actions.

B Baseline Training Settings

This appendix summarizes the baseline training settings used in the reported experiments. The environment-side reward, termination, and action-mask semantics follow the definitions in Section 3 and Appendix A.

Table 9: Common training and evaluation settings.

Setting	Value
Random seeds	0, 1, 2.
Episode horizon	Empty Goal: 200 environment steps; other front-goal scenarios: 300 environment steps; full-field benchmark: 3000 environment steps. These steps correspond to play_on server cycles.
Parallel environments	Reported main runs use 4 front-goal environments and 10 environments for the 11-vs-11 benchmark.
Training horizon	Empty Goal and Blocked Shot: 3M environment steps; other front-goal scenarios: 5M environment steps; full-field benchmark: 30M environment steps.
Evaluation	50 episodes per evaluation.
Evaluation/checkpoint interval	Front-goal scenarios: every 1M environment steps; full-field benchmark: every 10M environment steps.

Table 10: Baseline algorithm architectures.

Item	QMIX	MAPPO	ParaDQN
Action space	Base discrete action space, 19 actions.	Base discrete action space, 19 actions.	Hybrid parameterized action space, 6 discrete commands with continuous parameters.
Network architecture	Shared recurrent agent: Recurrent FC(o, a_{t-1}, i)-64, ReLU, with shared actor and centralized GRUCell-64, Q head; QMixer critic; MLPBase 128×2 with parameter vector; embed 32 with two-layer hyper-networks and hidden size 64.	MAPPO/RMAPPO Q network: MLP [256, 256] over observation and full action-MLPBase 128×2 with parameter vector; ReLU and LayerNorm; GRU-128; categorical action head.	actor: MLP [256, 256] with sigmoid-scaled continuous outputs.
Parameter sharing	Shared agent network across controlled players with centralized monotonic value mixing.	share_policy = True: one shared parameter actor with centralized critic.	One shared Q network and one shared parameter actor across controlled players.

Table 11: Baseline optimization and evaluation settings.

Item	QMIX	MAPPO	ParaDQN
Optimizer and learning rate	RMSProp, learning rate 5×10^{-4} , $\alpha = 0.99$, $\epsilon = 10^{-5}$.	Adam; actor learning rate 3×10^{-4} , critic learning rate 5×10^{-4} , $\epsilon = 10^{-5}$.	Adam; Q learning rate 10^{-4} , actor learning rate 2×10^{-4} .
Discount and returns	$\gamma = 0.99$.	$\gamma = 0.995$, GAE $\lambda = 0.98$.	$\gamma = 0.99$, 5-step return.
Batch and replay	Replay buffer of 64 episodes; warmup 32 episodes; batch size 16 episodes; 2 learner updates per collection batch.	On-policy rollout buffer with length equal to the environment episode limit; PPO epochs 5; minibatches 2; recurrent chunk length 10.	Replay capacity 100,000 team transitions; batch size 64; train every environment step after the buffer contains at least 64 transitions; one update per environment step by default.
Target update	Target network update every 20 episodes; Double Q enabled.	Not applicable.	Double Q enabled; Polyak update with $\tau_Q = 0.003$ and $\tau_\mu = 0.001$; actor update every 2 Q updates.
Exploration	Epsilon-greedy, annealed from 1.0 to 0.05 over 100k environment steps.	Stochastic policy during training.	Epsilon-greedy, annealed from 1.0 to 0.05 over 1M environment steps.
Loss and stabilization	TD loss with masked padded data; gradient clipping at 10.	PPO clipping coefficient 0.2; entropy coefficient 0.05; value-loss coefficient 1; Huber value loss; ValueNorm; maximum gradient norm 5; output gain 0.1.	Huber TD loss; gradient clipping at 10; actor behavior L2 coefficient 0.01; priority-biased replay with $\alpha = 0.4$.
Action-mask handling	Invalid actions are masked before Q maximization and action selection.	Invalid logits are set to -10^{10} before categorical sampling or evaluation.	Random exploration samples valid actions; invalid Q values are set to $-\infty$ for Q and target-Q selection; actor loss excludes non-parameterized and fallback actions.
Evaluation policy	Greedy policy.	Deterministic policy.	Target policy.

C Detailed Front-Goal Results

Table 12: Detailed front-goal results. Values are goal rates reported as mean $\pm$ standard deviation over three random seeds. Bold means indicate the highest nonzero mean within each row.

Scenario	Model	MaxEPV+mask	Mask only	MaxEPV only	No EPV/mask
Empty Goal	QMIX @3M	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.67 $\pm$ 0.47
	MAPPO @3M	0.94 $\pm$ 0.08	0.96 $\pm$ 0.03	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00
	ParaDQN @3M	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00
Blocked Shot	QMIX @3M	0.88 $\pm$ 0.02	0.87 $\pm$ 0.07	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	MAPPO @3M	0.66 $\pm$ 0.03	0.69 $\pm$ 0.08	0.89 $\pm$ 0.02	0.86 $\pm$ 0.05
	ParaDQN @3M	0.28 $\pm$ 0.07	0.02 $\pm$ 0.02	0.03 $\pm$ 0.02	0.14 $\pm$ 0.18
Support Option	QMIX @5M	0.28 $\pm$ 0.40	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	MAPPO @5M	0.71 $\pm$ 0.09	0.68 $\pm$ 0.05	0.83 $\pm$ 0.11	0.00 $\pm$ 0.00
	ParaDQN @5M	0.02 $\pm$ 0.03	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01
Passing Lane	QMIX @5M	0.69 $\pm$ 0.07	0.02 $\pm$ 0.02	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	MAPPO @5M	0.55 $\pm$ 0.05	0.53 $\pm$ 0.05	0.60 $\pm$ 0.03	0.00 $\pm$ 0.00
	ParaDQN @5M	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
Compact Defense	QMIX @5M	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	MAPPO @5M	0.32 $\pm$ 0.09	0.08 $\pm$ 0.11	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	ParaDQN @5M	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00

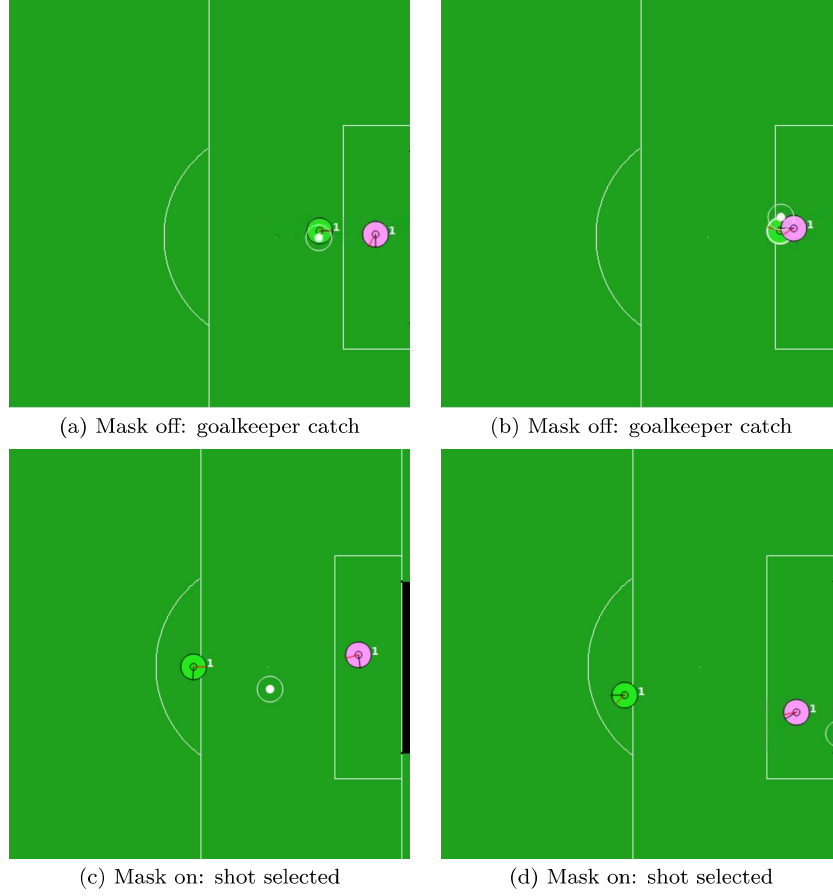


Figure 8: Qualitative QMIX examples in the Blocked Shot scenario. Green denotes the controlled attacker, pink denotes the goalkeeper, and the white circle indicates the ball. Without action masks, the policy may continue selecting dribbling or forward-movement actions until the ball enters the goalkeeper’s catchable area; with the shoot-priority mask, competing actions are removed and shooting is selected when executable.

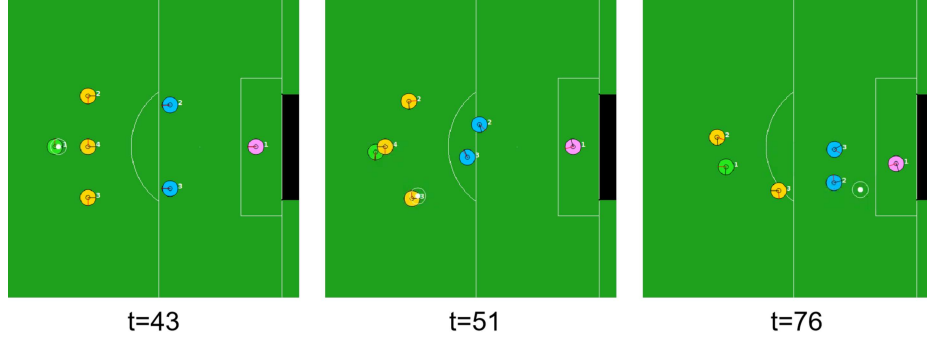


Figure 9: Qualitative MAPPO rollout in Compact Defense. Yellow and green players denote MAPPO-controlled attackers, blue players denote HELIOS Fallback defenders, the pink player denotes the goalkeeper, and the white circle indicates the ball. The sequence is from the Max-EPV+mask setting, the best-performing Compact Defense configuration.

D Additional Full-Field Results



Figure 10: 11-vs-11 full-field return curves over 30M environment steps. Each panel shows one baseline algorithm, and each curve corresponds to one random seed.

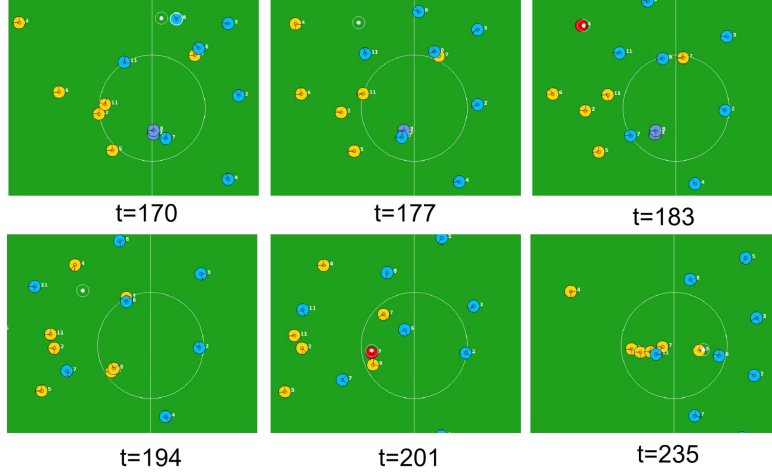


Figure 11: Qualitative MAPPO rollout example in the 11-vs-11 full-field benchmark. Yellow players are MAPPO-controlled left-team agents, blue players are HELIOS Fallback opponents, and the white circle indicates the ball. The selected frames illustrate ball recovery, a pass, and subsequent ball progression toward the opponent half during one evaluation episode.

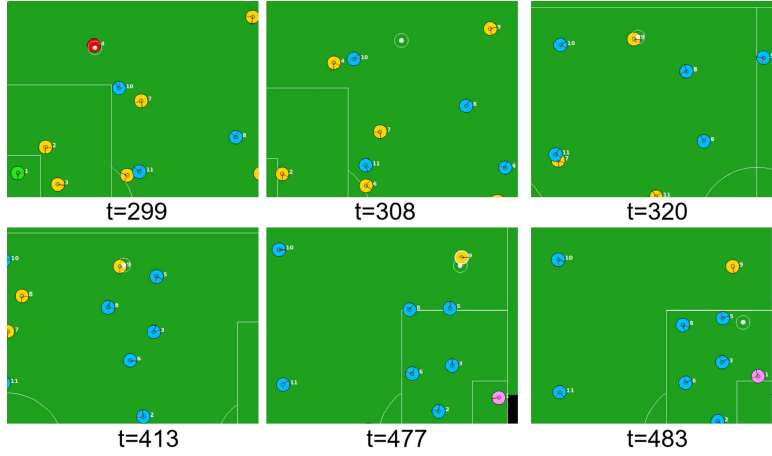


Figure 12: Qualitative QMIX rollout example from a high-performing seed in the 11-vs-11 full-field benchmark. Yellow players are QMIX-controlled left-team agents, blue players are HELIOS Fallback opponents, and the white circle indicates the ball. The selected frames illustrate passing, ball carrying toward the side of the opponent penalty area, and an unsuccessful shot attempt.

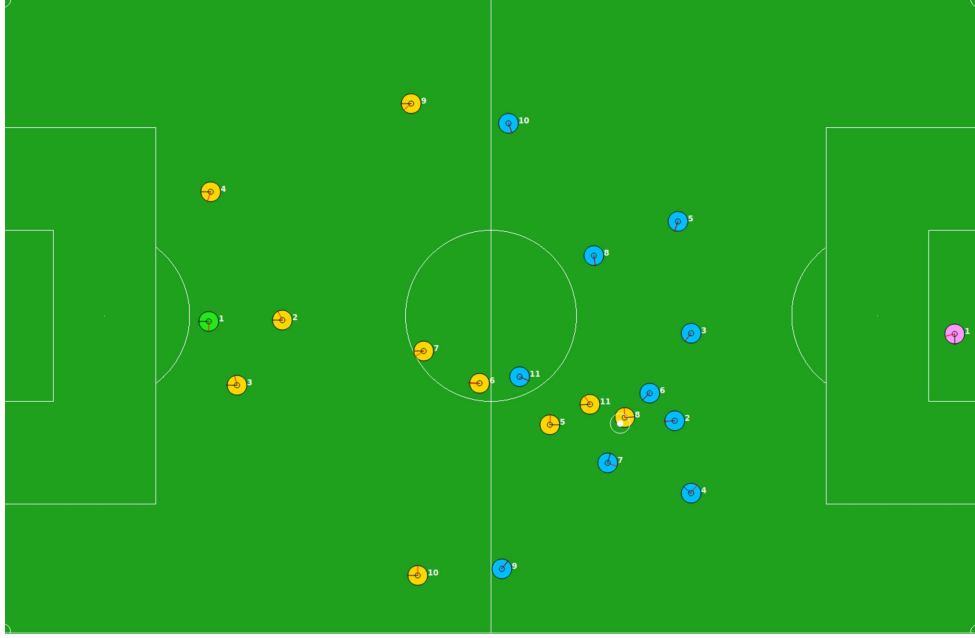


Figure 13: Qualitative off-ball movement example in the 11-vs-11 full-field benchmark. Yellow players are controlled left-team agents, blue players are HELIOS Fallback opponents, and the white circle indicates the ball. The visualization shows that, except for players close to the ball, some off-ball players far from the ball continue moving in fixed directions and do not yet form a coherent team shape.